

What is Firebase?

Google's Firebase is a complete Backend as a Service (BaaS) solution. It provides developers with a set of cloud based tools and infrastructure that allows them to save from developing or managing their own servers, databases or authentication systems. As applications are connected directly to Firebase via official SDKs and REST APIs, Google takes care of the infrastructure, scaling and availability automatically. In 2011, Firebase was first released as a real time database product, and was acquired by Google in 2014. It has since become a full fledged development platform, powered by Google Cloud infrastructure and enterprise class reliability, global data distribution and integration with Google Analytics and Google Cloud Functions (Firebase, 2026).

Core Services?

Firebase is a collection of cloud services. Cloud Firestore is the main database, which is a NoSQL document store that synchronises data in real time with all connected clients. Firebase Authentication manages the identity of the user, including email/password authentication, Google Sign In, and multiple OAuth providers. Firebase Cloud Messaging (FCM) is a service that enables web and mobile devices to receive push notifications. Trusted server processes have complete programmatic access to all Firebase services through the Firebase Admin SDK. Web content can be both static and dynamic (Firebase Hosting), and uploaded files can be used by the users (Firebase Storage). We are currently using Sentinel Guard for Cloud Firestore (all data storage), Firebase Authentication (user login, JWT tokens) and the Admin SDK (used by the Python backend and the FastAPI server). Push notifications are a feature that is planned for FCM, and can be configured (Firebase, 2026).

How Firebase Works?

Firebase is a managed cloud bridge between application clients and back end services. Each Firebase project is a logical container for all of the services, credentials, and configuration. Each service (Firestore, Auth, FCM) will have its own context and rules environment, but can share authentication for the project (*Firebase Documentation*, 2025).

Architectural Model

The client applications, including the Sentinel Guard React dashboard, communicate and connect with the server using the Firebase JavaScript SDK, initialized with a `firebaseConfig` object containing the project's API key, auth domain, and project ID. They are public keys used to identify the project and not used to make access unrestricted, all access is through Firebase Authentication and Firestore Security Rules. The Python server and FastAPI backend utilize the Admin SDK and are authenticated using a private service account key and have full privileges without using Security Rules. The two tier trust model makes it impossible for client code to go past the rules and for the server code to directly access the database.

Firestore Data Model

Unlike tables and rows, Cloud Firestore stores data in Collections and Documents. A collection is a group of documents that are collected together, like a table in a database with a named. Document one record with key value fields, similar to a row.

Subcollections can be used to represent hierarchical data within documents. The values in fields are typed; strings, numbers, booleans, timestamps, arrays, and maps.

Sentinel Guard uses six top level collections. Every validated sensor data, logging of temperature, humidity, packet status, and ML classification readings by the python server into the `sensor_data` collection. Security events and alerts are collected in the `security_events` and `alerts` collections, and represent threats that are detected, such as tampered packets, replay attempts, DDoS floods, and false data injections. Operational server logs are in `system_logs` collection. Each user's email, display name and role are stored in the `users` collection. The `roles` collection contains role definitions which are only available to admins.

Real Time Data Flow

The ESP32 sensor uses AES 128 encryption to create a data payload and adds an HMAC SHA256 signature to it, then it publishes the encrypted payload to the Mosquitto MQTT broker, using TLS. The Python server subscribes to the broker, checks the signature, decryption of the payload, then executes the complete security pipeline: DDoS check, structure validation, replay detection, range check, and ML anomaly classification. When all of the checks pass, the server writes the reading to Firestore using the Admin SDK. The React dashboard is connected to `sensor_data` via an `onSnapshot()` listener, and as soon as a new document is written, Firestore pushes the update to any connected browser clients, and the dashboard will re render in real time. If a packet is not admitted to the security system, it will not be included as a valid sensor packet, but will be logged as a threat in `security_events`.

How to Connect to Firebase

Project Configuration

All Firebase integrations start from the Firebase Console at console.firebase.google.com. The console produces a `firebaseConfig` object after creating a project and registering a web app. The Sentinel Guard project details are: Project ID `sentinel-guard-5c77c`, Project Number `758190988508`, App ID `1:758190988508:web:48228997edd04c1ecca819`, Auth Domain `sentinel-guard-5c77c.firebaseio.com`, and Measurement ID `G-Q12K16PTKT`. These values are simply copied into the frontend code.

Frontend (React Dashboard)

The React dashboard includes the Firebase JS SDK in the project using `npm install firebase` and generates an initialisation file called `firebase.js`. Firebase is initialized once with `initializeApp(firebaseConfig)` and services are retrieved like `getAuth(app)` and `getFirestore(app)`. The occurrences are exported and imported throughout the React component tree as necessary for state of authentication and database access. To read the data in real-time, the dashboard will call `onSnapshot()` on the collection you want to read, passing a callback that will be called whenever a document is added or changed. This one call can be used instead of a polling loop, a REST endpoint, and a WebSocket connection.

In the frontend, the `signInWithEmailAndPassword(auth, email, password)` function is being used for authentication. On success, Firebase will return an ID Token (a signed JWT containing the UID, email and any custom claims defined by the server). Each time the call is made to the FastAPI backend, this token is attached to it and verified on the backend using the `auth.verify_id_token()` function.

Backend (Python Local Server)

The Python server and FastAPI user management service both connect via the Firebase Admin SDK, installed with `pip install firebase-admin`. A service account key, which is a JSON credentials file downloaded from Project Settings > Service Accounts in the Firebase Console, is needed for initialisation. This file should never be added to version control, and should remain secret. Once the Admin SDK is initialised at startup using `firebase_admin.initialize_app(credentials)`. Afterwards, `db = firestore.client()` will give you full access to the database from the certificate('serviceAccountKey.json').

Each validated sensor reading is written to the Python server with `db.collection('sensor_data').add({...})` and a detected threat is added to the Python server with `db.collection('security_events').add({...})`. The FastAPI service uses `auth.create_user()`, `auth.set_custom_user_claims()`, and `auth.delete_user()` to manage the user lifecycle, and `db.collection('users').document(uid).set({...})` to keep Firestore in sync with Firebase Authentication.

Composite Indexes

Firestore needs to have composite indexes for any query that filters or sorts on more than one field. There are two indexes that are set up for Sentinel Guard. The `security_events` collection is indexed on `timestamp` (descending), `severity` (ascending), and `__name__` (ascending) which enabled the query in the alerts page, which filters for severity and sorts by time. A filtered historical query against the `sensor_data` collection is possible due to the following indexes: `timestamp` (descending), `status` (ascending), and `__name__` (ascending). Both indexes are activated, and come with an ideal 1.00 index entries to results ratio.

Authentication and Role Based Access Control

Firestore Authentication

Enabled sign in providers - Email/Password. As an example, There are currently 2 users registered, `r@1.com` (registered on 11 May 2026) and najathnajeem7@gmail.com (registered on 27 Apr 2026). Upon signing in, Firebase issues an ID Token which is a JWT with a 1 hour expiration and is automatically renewed by the SDK. This token is verified by the FastAPI backend on each admin API request, using the call to `auth.verify_id_token(token)`, which not only validates the token's signature, but also decodes the claims

Role Based Access Control (RBAC)

There are two roles here, Admin and Standard. A compromise is made to balance correctness and performance by having roles in two locations. The users collection in Firestore is the authoritative store, every Firestore user document has a role field of 'admin' or 'standard'. This is updated using the PUT `/admin/users/{id}/role` endpoint of FastAPI. The role is also embedded as a custom claim in the user's JWT, so that Firestore Security Rules can access the role directly from the `request.auth.token.role` without having to make extra queries to Firestore for every rule evaluation.

The endpoints listed below are all of the admin only endpoints exposed by the FastAPI service, which are protected by the `get_current_admin()` dependency. GET `/admin/users` returns all the users stored in Firestore. This POST `/admin/users` operation adds a new user to Firebase Auth, assigns custom claims to the user, and documents the user's profile in Firestore. Updates role in Firestore and custom claims atomically. DELETE `/admin/users/{id}` deletes the user both in Firebase Auth and Firestore. Each endpoint checks the caller's token and checks if the caller is an admin before continuing.

How to use Firestore Security Rules

Firestore Security Rules are encoded in a declarative language that is customized for them. They're created in the Firebase Console under Firestore Database > Rules, or in a `firestore.rules` file. Deployment from the CLI requires `firebase deploy only firestore:rules`, and will take effect in about 60 seconds. The rules are checked on the server and deny everything by default, if there is no rule in the collection, it is not accessible (*Firebase Documentation*, 2025).

Rule Structure

A service block is a rule wrapped to a Cloud Firestore service, and match blocks contain the rules. A match block can set a document path (wildcards are used in order to match any document in a collection), and allow statements within the block specify the operations that are allowed and on which conditions. The 5 operation keywords are read (get operations on a single document and list operations on a collection), write (create, update, and delete operations), create, update, and delete. With granular keywords you can have more control the security_events can be created and updated by the admin, and delete is set to if false, meaning that no security event record can ever be deleted from Sentinel Guard.

```
37
38
39 // SENSOR DATA
40
41 match /sensor_data/{docId} {
42   allow read: if isAuthenticated();
43   allow write: if false; // backend only via Admin SDK
44 }
45
46 // SECURITY EVENTS (includes alerts)
47
48 match /security_events/{docId} {
49   allow read: if isAuthenticated();
50
51   // Require all core fields to prevent malformed documents
52   allow create: if isAdmin() &&
53     request.resource.data.keys().hasAll([
54       'eventType', 'severity', 'description', 'timestamp'
55     ]);
56
57   // Only acknowledgement fields can change — event itself is immutable
58   allow update: if isAdmin() &&
59     request.resource.data.diff(resource.data)
60       .affectedKeys().hasOnly(['acknowledged', 'acknowledgedBy', 'acknowledgedAt']);
61
62   allow delete: if false; // permanent audit records
63 }
64
```

Note: This Picture show the firebase firestore rules

Helper Function and the Requested Object

Rules can contain reusable functions which are used by Sentinel Guard to prevent identical logic in all match blocks. If request.auth is not null, the isAuthenticated() function will return true, which is helpful for knowing that the user is signed in. The getUserRole() function first pulls the user role claim from the user's JWT token (request.auth.token.get('role', ...)), and if it doesn't exist, it pulls the user's Firestore document as a fallback in order to process cases where the JWT token was issued before the role was granted. isAuthenticated() is the combination of isAdmin() and isStandard() with a role check. isOwner(userId) tests whether the document is owned by the user specified by the userId.

```
// HELPER FUNCTIONS

function isAuthenticated() {
  return request.auth != null;
}

// JWT claim is faster; Firestore fallback handles stale tokens
function getUserRole() {
  return request.auth.token.get('role',
    get(/databases/${database}/documents/users/${request.auth.uid}).data.role
  );
}

function isAdmin() {
  return isAuthenticated() && getUserRole() == 'admin';
}

function isStandard() {
  return isAuthenticated() && getUserRole() == 'standard';
}

function isOwner(userId) {
  return isAuthenticated() && request.auth.uid == userId;
}
```

Note: This picture shows the Helper function in firestore rules

With any inside rule conditions, there are always two data objects available. The request object includes the authenticated user context (request.auth.uid, request.auth.token), and request.resource.data includes the document's field values as they will be stored in the document after it is written for incoming data to validate the write. The resource object contains the state of the document being edited just prior to writing, and is passed to resource.data.diff() to limit what can be changed in the document during the update.

Self Update Restriction and Field Validation

The users collection rule demonstrates how to enable a user to update their profile, while not being able to raise their role. The .diff().affectedKeys().hasOnly([...]) pattern calculates the difference between the incoming and existing document and makes sure only the specified fields are changing. If the client attempts to modify the role field, the diff will contain 'role' and the rule will block the modification of the role field.

```
// USERS

match /users/{userId} {
  // Users can read their own profile, admins can read any
  allow read: if isAuthenticated() && (isOwner(userId) ||
    get(/databases/${database}/documents/users/${request.auth.uid}).data.role == 'admin');

  allow create, delete: if isAdmin();

  // Non admins can only update display name and phone (not role)
  allow update: if isAdmin() ||
    (isOwner(userId) &&
      request.resource.data.diff(resource.data)
        .affectedKeys().hasOnly(['displayName', 'phoneNumber']));
}
```

Note: The picture of User function in Firestore Rules

Testing and Common Pitfalls

The Rules tab of the Firebase Console has Rules Playground to simulate any operation (get, list, create, update, or delete) against the rules without affecting any real data. A test defines the document path and the simulated user's UID and custom claims, and optionally, the document data. During testing of unexpected denials it is easy to see what rule line caused that by looking at the playground.

The following are common pitfalls when writing Firestore rules: include allow read, write: if true in development and forget to lock down the rule before deployment; only include frontend checks and forget to include firestore level checks; use get() within the rules without a custom claim from the JWT which results in an additional billable read per rule evaluation; and not have a rule for a collection at all that will result in Firestore silently denying all access and lead to confusing errors in the app.

Firebase Summary

Firebase Service	Used By	Role
Cloud Firestore	Python Server + React Dashboard	Pushes sensor readings, security events, users, logs and audit records to clients in real-time
Firebase Auth	React Dashboard + FastAPI	Email/password login; issues and verifies JWTs with user identity and role claims
Admin SDK (Python)	Python Server + FastAPI	Full server side database access, user lifecycle management, token verification
Firebase JS SDK	React Dashboard	Client side auth, realtime onSnapshot() listeners, live data rendering